

An Agentic Large Language Model
Copilot for Real-Time Aircraft
Turnaround Delay Attribution and
Recovery

By

Seif Eldin Tourky

Submitted to

The University of Roehampton

In partial fulfilment of the requirements

for the degree of

Master of Science

in

Artificial Intelligence

Declaration

I hereby certify that this report constitutes my own work, that where the language of others is used, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of others.

Generative AI tools were used during the preparation of this project to assist with drafting and editing prose, and to assist with software development. The research design, operational domain model, implementation decisions, evaluation methodology, results and analysis are the author's own work. All AI-assisted text and code were reviewed, tested and revised by the author, who takes full responsibility for the content of this report.

I declare that this report describes the original work that has not been previously presented for the award of any other degree of any other institution.

Seif Eldin Tourky

01/08/2026

Seif Eldin Tourky

Acknowledgements

I would like to take the opportunity to thank my family, and the few people close to my heart. Thank you for believing in me, for the patience, the encouragement, and the sacrifice that each and every one of you has made with my best interest at heart. It has been a wonderful story, and after all the sweat, blood and tears, it is only my pleasure to dedicate this to those who believed when it would have been easier not to, and who showed up when it was never owed.

The way here has been a challenging one. It has crossed lives and countries and more than one version of what my future was supposed to look like. Beginnings were started over and backgrounds left behind. What never changed was the intention. I have always meant to build something of value, and I have never questioned what it would cost. So, to myself as well: thank you for holding that line, through every change of circumstance, every early morning after a late shift, and every version of this that had to be begun again.

To my brother, who has been my example of what persistence looks like in practice. I did not need to be told that hard work pays off; I watched it. On the days this project felt beyond me, the memory of how you keep going was often what got me back to the desk.

To my mother, whose support has never once come with a condition attached. You believed in this before there was anything to believe in.

And to my father, who taught me perseverance before I had any idea what it was for. You gave me the stubbornness to finish things. This is one of them.

I am also grateful to my supervisor for the guidance and direction given throughout this project.

To all of you, Thank you. This is yours as much as it is mine.

Abstract

Aircraft turnaround coordination is conducted under time pressure against a fixed departure, by a coordinator who is accountable for the departure but commands none of the teams involved. The monitoring tools available raise an alert once a milestone target has already been exceeded, by which point the opportunity to intervene has passed, and they offer no account of why a turnaround is running late or what should be done about it. Research attention has concentrated on predicting departure times more accurately, which addresses a different problem from the one a coordinator faces.

This project designs, implements and evaluates an agentic large language model system that monitors a turnaround in real time, attributes the cause of developing delay, and recommends explainable recovery actions to a coordinator who retains the decision. The domain model was elicited from operational practice and from real dispatch records, and formalises the backward countdown from departure, the rebasing of the schedule at arrival, and a three-level decision hierarchy in which safety is an absolute constraint, proactive action is mandatory, and minute recovery is subordinate to both. The reasoning component is interchangeable between a rule-based backend and a language model, and every output passes through a deterministic constraint layer that enforces the hierarchy by construction rather than by instruction.

The central formulation distinguishes between the moment when a delay cause becomes knowable and the later moment when elapsed time indicates that the target will be missed. The interval between the two is the window in which an agentic system can act, and a critical-path method cannot, because such a method has no representation of cause.

Across 86 simulated breaches sampled from real delay occurrences, the system identified a developing breach a mean of 23.8 minutes before door closure, against 19.8 minutes for a critical-path projector and zero for a threshold dashboard and attributed a cause on all 86 occasions where neither baseline attributed any. On breaches whose causes admit intervention, an explicit recovery model estimates that it would recover 19.1 per cent of the delay, a result robust to sensitivity analysis on its principal assumption; on those that do not, it recovers nothing but produces directed actions in every case. The advantage is exactly zero where a cause becomes knowable only when it becomes provable, a limit the formulation predicts and the evaluation confirms. An appraisal of all 27 comparison cases by a practising coordinator identified three

defects in the system's recommendations, each of which was corrected and the results regenerated.

The work contributes a formulation of the conditions under which agentic reasoning improves on classical projection, a decision hierarchy elicited from practice and enforced structurally, and an evaluation conducted against the operational decision the system supports rather than against statistical accuracy alone. Its principal limitation is that no recommendation has yet been acted upon in a live operation.

Table of Contents

Declaration	ii
Acknowledgements	iii
Abstract	iv
Table of Contents.....	vi
List of Figures.....	ix
List of Tables	x
Chapter 1 Introduction	1
1.1 Problem Description, Context and Motivation	1
1.2 Objectives	1
1.3 Methodology	2
1.4 Legal, Social, Ethical and Professional Considerations	3
1.5 Background.....	5
1.6 Structure of Report.....	6
Chapter 2 Literature – Technology Review	7
2.1 Literature Review.....	7
2.2 Technology Review	10
2.3 Summary.....	11
Chapter 3 Implementation	12
3.1 Architecture and Design Position	12
3.2 Operational Domain and Source Data	13
3.3 The Backward Countdown and the In-Block Anchor	14
3.4 Milestone Model and Attribution.....	15
3.5 The Three-Level Decision Hierarchy	16
3.6 The Signal–Proof Formulation	17
3.7 Swappable Reasoning and the Constraint Layer	18
3.8 Baselines	19

3.9	Interface	20
3.10	Implementation and Reproducibility	21
Chapter 4 Evaluation and Results		23
4.1	Evaluation Design	23
4.2	Detection: The Signal–Proof Interval	23
4.3	Outcome: Recovered Delay	25
4.4	Comparison of Reasoning Backends	26
4.5	Safety Behaviour and the Constraint Layer	28
4.6	Expert Appraisal and Design Revision	30
4.7	Threats to Validity	32
4.8	Related Works	33
Chapter 5 Conclusion		34
5.1	Summary of the Work	34
5.2	Achievement Against Aims	34
5.3	Comparison with the State of the Art	35
5.4	Strengths and Limitations	35
5.5	Future Work	36
5.6	Reflection	37
5.7	Conclusion	38
References		39
Appendices		I
Appendix A: Project Proposal		II
Appendix B: Project Management		III
Appendix C: Artefact/Dataset		V
Appendix D: Screenshot		VII
Appendix E: Appraisal Instrument		VIII

List of Figures

Figure 1 System architecture. The projector is invoked by the agent as a tool and also serves as an independent baseline; every reasoning output passes through the constraint layer.

Figure 2 Rebasing from the in-block anchor. Late inbound time is absorbed into the rebased door target rather than counted as a breach.

Figure 3 Safety hold with manage-around actions retained beneath it. Safety takes priority in action, not in attention.

Figure 4 The signal–proof interval, which defines the window in which an agentic system can act, and a critical-path method cannot.

Figure 5 The demonstration interface at ten minutes before door closure. The agent has attributed the delay and issued recovery actions; the critical-path projector has not yet called the breach.

Figure 6 Regression suite output. Eleven constraint-layer invariants driven by adversarial reasoners, plus two schedule regressions against the recorded dispatch records.

Figure 7 Detection lead and coverage against both baselines across 86 simulated breaches.

Figure 8 Project Gantt chart. Eight work packages tracked from domain elicitation through to the written report, revised twice as scope developed.

List of Tables

[Table 3.1 Principal components and their responsibilities.](#)

[Table 3.2 Turnaround profiles. Preparation windows vary by aircraft variant where no crew change occurs.](#)

[Table 4.1 Detection lead time by approach, 40 simulated days \(86 breaches\). Standard deviations are population values.](#)

[Table 4.2 Detection advantage decomposed by delay code, measured as agent lead time minus critical-path lead time.](#)

[Table 4.3 Delay recovered on breaches with recoverable causes \(n = 35 of 86\), mean delay 14.2 minutes, recovery rate \$r = 0.15\$.](#)

[Table 4.4 Agreement between the rule-driven and language-model reasoners across 27 cases.](#)

[Table 4.5 Behaviour on safety-flagged probe cases. All four returned a safety hold with a non-empty set of accompanying actions.](#)

[Table 4.6 Expert appraisal of agent recommendations across the 27 evaluation cases \(single respondent\).](#)

[Table 4.7 Division of responsibility between the deterministic components and the interchangeable reasoning backend.](#)

[Table 5.1 Objectives from Section 1.2 against outcomes.](#)

Chapter 1 Introduction

Every commercial flight spends part of its day on the ground, and the interval between arrival and departure is one of the few points in a flight's cycle where lost time can still be recovered. That interval is coordinated by a single person working against a fixed departure time, across several service providers, usually with nothing more than a handheld timestamp application and a monitoring tool that raises an alert once a deadline has already passed. This project asks whether an agentic system built on a large language model can do something those instruments cannot: identify why a turnaround is running late while there is still time to act and recommend what the coordinator should do about it. The chapter that follows states the problem, the objectives and the methodology, sets out the legal, social, ethical and professional considerations that shaped the design, provides the operational background, and describes the structure of the report.

1.1 Problem Description, Context and Motivation

Existing decision support for turnaround coordination predicts times without attributing causes and raises alerts without recommending actions [3], [4], [7]. A prediction that a departure will be late, unaccompanied by an explanation of why and a proposal for what to do, transfers no useful information to a coordinator who can already see that the turnaround is running behind. The problem addressed by this project is therefore not prediction accuracy but decision support: identifying the cause of a developing delay early enough for intervention to remain possible and recommending an action that a coordinator would recognise as appropriate.

1.2 Objectives

The aim of this project is to design, implement and evaluate an agentic large language model system that monitors an aircraft turnaround in real time, attributes the cause of developing delay, and recommends explainable recovery actions to a human coordinator who retains the decision. The objectives are as follows.

- Elicit and formalise the operational model of turnaround coordination, including the backward countdown, the rebasing of the schedule at arrival, and the decision hierarchy that practitioners apply.

- Implement an agentic architecture that perceives a milestone event stream, projects the outcome, attributes cause and recommends action, with the reasoning component interchangeable so that its contribution can be isolated [16].
- Guarantee safety-relevant behaviour by construction rather than by instruction, through a deterministic constraint layer applied to every reasoning output [23].
- Evaluate the system by comparing it to the alternatives it claims to improve upon, specifically threshold monitoring and classical critical-path projection, while measuring detection lead time, cause attribution, and recovered delay [1], [12].
- Appraise the operational validity of the system's recommendations with practising coordinators and revise the system in response.

The project is organised around a single comparative question: what does an agentic reasoning layer add over classical scheduling and rule-based baselines in turnaround delay recovery, and where does it fail? The question is deliberately two-sided, since a contribution that cannot state its own limits is not established but merely asserted.

Four contributions are claimed. The first is the signal–proof formulation, which distinguishes the moment a delay cause becomes knowable from the later moment at which elapsed time proves the target will be missed, and identifies the interval between them as the window in which an agentic system can act and a critical-path method cannot. The second is a three-level decision hierarchy elicited from practice, in which safety is an absolute constraint, proactive assertion is mandatory, and minute recovery is subordinate to both. The third is a constraint layer that enforces this hierarchy deterministically, converting it from behaviour a language model usually exhibits into a property that the system guarantees [23], [24]. The fourth is an evaluation that measures the system against the decision it supports rather than against statistical accuracy alone, and that reports where the approach yields no advantage as well as where it does.

1.3 Methodology

The project follows a design-science approach, in which an artefact is constructed to address a practical problem and is evaluated against the problem it was built to solve [39]. The approach

was selected over a purely experimental design because the contribution claimed is architectural rather than statistical, and over a purely descriptive study because the central question concerns what a system can do that existing instruments cannot.

Domain knowledge was elicited from operational practice and from real dispatch records collected in a working ground-handling operation and formalised into an executable model of the turnaround countdown [1], [2]. The system was then implemented incrementally, with the reasoning component deliberately isolated behind a fixed interface so that alternative strategies could be substituted and compared.

Evaluation proceeds on four fronts. Detection performance is measured by discrete-event simulation over scenarios sampled from the real delay occurrences, comparing the system against two implemented baselines representing the instruments currently in operational use [1], [12]. Outcome performance is estimated under an explicit recovery model whose principal parameter is subjected to sensitivity analysis rather than asserted. The contribution of the reasoning layer is isolated by comparing interchangeable backends on an identical case set, an approach consistent with the evaluation practice surveyed for agentic systems [24]. Operational validity, which simulation cannot establish, is assessed by appraisal with a practising coordinator, and the system revised in response to the defects identified.

Correctness is maintained by an automated regression suite covering the schedule arithmetic against the real records and the invariants of the constraint layer, the latter driven by adversarial inputs. Work was tracked against a work-package plan with a Gantt chart, revised as scope developed, and the implementation was version-controlled throughout. The technologies selected, and the reasoning behind each choice, are set out in Section 2.2.

1.4 Legal, Social, Ethical and Professional Considerations

The project operates in a live commercial aviation context and raises considerations in four areas, each of which shaped design decisions rather than being addressed retrospectively.

Legal. The dispatch records from which the domain model was elicited are the commercial property of the ground-handling organisation concerned and are subject to contractual confidentiality. They are not reproduced in this report, are excluded from version control, and are held only in a private repository. A de-identified derivative containing delay codes, durations and discovery points, with all operator, airport, date and personnel references removed, was extracted

for evaluation, and the evaluation was verified to reproduce every reported figure with the confidential files absent. The organisation and the carrier are referred to generically throughout the report. The records contain operational timestamps rather than personal data and consequently do not engage the UK General Data Protection Regulation; where individuals appear in the source material, such as a late flight crew member, they are represented in the system only as a delay code and never as an identifiable person. Aviation delay coding follows the IATA standard [1], which is applied descriptively rather than reproduced. No safety-regulated system is modified by this work: the artefact is advisory and sits outside the certified operational systems governed by the Civil Aviation Authority.

Social. A system that recommends actions to a coordinator has consequences for the people whose work those actions direct. Recommendations are phrased as coordination steps rather than as instructions to named individuals, and the system must remain incapable of becoming a mechanism for monitoring individual worker performance. This scenario is a foreseeable misuse of any tool that timestamps ground handling activity with attributed causes [14], [15], and it is recorded here as a constraint on legitimate deployment. Delay attribution in this domain also carries commercial consequence, since codes determine cost allocation between carrier and handler; the system's attributions are therefore presented as inferences for the coordinator to confirm, never as an authoritative record. Accessibility considerations arising from the operational context, including passengers with reduced mobility and enhanced medical assistance, are represented explicitly in the milestone model rather than treated as exceptions, and the appraisal reported in Chapter 4 led to the medical case being revised to give clearer priority to the passenger's welfare over the schedule.

Ethical. Two ethical issues arise. The first concerns the respondents in the appraisal reported in Section 4.6. Participation was voluntary and anonymous, no name, staff number or employer was recorded, the instrument transmitted nothing automatically, and respondents were informed of the purpose before beginning. The instrument was constructed to be low-risk by design for this reason. The exercise was conducted as a professional appraisal of system outputs on synthetic scenarios rather than as research on human subjects, and formal ethical clearance was accordingly not sought. No personal data, identifying information or data about the respondents themselves was collected; the instrument recorded only their professional assessment of the system's recommendations. Had the exercise been designed to study the respondents rather than the system, clearance would have been required, and the properly constituted user study proposed in

Section 5.5 would be conducted under it. The second concerns the position of the system with respect to human judgement. The literature on automation bias establishes that confident recommendations are accepted uncritically under time pressure [36], [37], which is precisely the condition in which this system operates. The design response is that the system advises and never acts, that reasoning is exposed for verification rather than concealed behind a conclusion [16], and that the interface presents recommendations as a checklist the coordinator may reject. A limitation of the appraisal, declared in Chapter 4, is that its principal respondent is also the author of the system.

Professional. The work is conducted in accordance with the BCS Code of Conduct, particularly the requirements to have regard for public health and safety, to work within one's competence, and to avoid misrepresenting the capability of a system [40]. Three consequences follow. Safety-relevant behaviour is enforced structurally rather than by instruction, because a component governing safety should not rest on a statistical tendency [23]. Claims are bounded to what the evidence supports: a single intervention by the constraint layer is reported as a single instance and not as a rate, and a facility added late in development is explicitly excluded from the evaluated results. Defects discovered during development, including a measurement error that initially produced a below-chance result, are disclosed rather than omitted. The author's dual position as both developer and practising coordinator in the operation studied is a source of both the domain access that made the project possible and a conflict of interest that is declared wherever it bears on the findings.

1.5 Background

An aircraft turnaround is the interval between arrival and departure during which disembarkation, cleaning, refuelling, catering, loading and boarding must be completed against a fixed departure time [1], [2]. The work is executed by several independent providers and coordinated by a turnaround coordinator who holds responsibility for the departure but commands none of the teams directly. Coordination is conducted under time pressure, across radio and telephone, and frequently over several aircraft at once.

The instruments available to the coordinator are limited. Milestone timestamps are recorded in a handheld application, and monitoring tools raise an alert once a target has already been exceeded. Such an alert arrives after the opportunity to act on it has passed. The judgement that matters operationally, namely whether a developing situation still admits intervention and what

that intervention should be, rests entirely on the coordinator's experience and is supported by no tool.

1.6 Structure of Report

Chapter 2 reviews the literature on turnaround prediction, delay propagation, automated sensing, agentic language models and human factors, and reviews the technologies selected. Chapter 3 describes the implementation, including the operational domain model, the decision hierarchy, the signal–proof formulation and the constraint layer. Chapter 4 reports the evaluation against both baselines, the comparison of reasoning backends, the expert appraisal and the revisions it produced, and the threats to validity. Chapter 5 concludes, evaluates the work against its aims, identifies future work and reflects on the process.

Chapter 2 Literature – Technology Review

This chapter establishes the academic and technical context for the project. It reviews five bodies of work: the collaborative decision-making frameworks that structure how turnaround events are recorded, the machine learning literature on turnaround and delay prediction, work on delay propagation and its absorption, automated sensing of ground service actions, and agentic large language model architectures together with their documented failure modes. It closes with the human factors literature on automation and trust, which constrains how a system of this kind may legitimately be positioned. The technology review that follows justifies each technology selected by reference to those findings rather than by preference, and the summary states the gap the project addresses and the methodological commitments that follow from it.

2.1 Literature Review

The aircraft turnaround is the interval between arrival and departure during which disembarkation, cleaning, refuelling, catering, loading and boarding are executed by separate providers against a shared deadline. Airport Collaborative Decision Making (A-CDM) structures this work as a sequence of milestones shared between stakeholders, with the objective of improving predictability through common situational awareness [1], [2]. A-CDM defines what should be recorded and when but is a data-sharing framework rather than a decision-making one: it does not determine what a coordinator should do when a milestone is missed.

A substantial body of work applies machine learning to turnaround and departure prediction. Approaches include cascaded gradient boosting over milestone times [3], fusion of turnaround sub-processes to estimate ground time [4], arrival-time prediction for ground resource allocation [5], and forecasting from synthetic aviation data [6]. Delay prediction more broadly has been addressed with conventional classifiers [7], spatio-temporal models with causal correction [8], and chained prediction architectures [11]. Three characteristics recur across this literature and bear directly on the present work. First, the prediction target is almost invariably a time or a binary delay indicator, not a cause. Second, evaluation is conducted against held-out historical data using statistical accuracy measures, with no assessment of whether a prediction would have altered an operational decision. Third, the predicted quantity is typically the outcome for a flight rather than the intermediate state of an in-progress turnaround, which limits applicability to real-time coordination.

These approaches differ in ways that are consequential but rarely compared directly. Tree-ensemble methods such as those in [3] and [7] offer interpretable feature importances and tolerate the modest, heterogeneous datasets typical of ground operations, but assume the features available at inference match those at training, an assumption that fails when a station changes its recording practice. The spatio-temporal and graph-based models of [8] and [9] capture network effects that tree ensembles cannot, but they require network-wide data that a single handler lacks and cannot obtain. Sub-process fusion [4] is the closest in spirit to the present work because it decomposes the turnaround instead of treating it as a single duration; however, it fuses sub-process estimates into an aggregate rather than preserving the decomposition as the output, which is the specific information a coordinator needs. A more fundamental limitation is shared across the group: evaluation is uniformly by error metrics against held-out data, and no study reviewed reports whether a prediction changed a decision or improved an outcome. Accuracy is thereby treated as a proxy for usefulness without that relationship being tested, and a model that is accurate but unactionable is indistinguishable, under these metrics, from one that is both.

A related strand models how delay propagates along aircraft rotations and how it might be absorbed. Work in this area includes day-ahead routing informed by data-driven primary delay predictions [12], propagation forecasting under extreme weather [9], and buffer optimisation that integrates machine learning with scenario analysis [13]. This literature marks a transition from prediction towards prescription, in that its output is a schedule or a buffer rather than an estimate. The prescriptions are nonetheless tactical and pre-departure: they adjust plans in advance rather than intervening in a turnaround already in progress.

A separate line of work addresses the sensing problem directly, using computer vision to detect and timestamp ground service actions automatically [14], an approach commercialised in operational products [15]. This work is complementary rather than competing: it improves the fidelity and timeliness of the event stream but does not interpret it. Determining what a late milestone means, whether it can be recovered, and what should be done in response remains outside its scope.

The literature on agentic large language models provides the mechanisms for automating such interpretation. ReAct interleaves reasoning traces with actions, producing behaviour that can be inspected rather than only observed [16]. Tool use allows a model to delegate computation it cannot reliably perform to external components [17], [18], a capability surveyed extensively and

directly relevant where numerical projection must be exact. Planning with language models has been examined both in classical planning formalisms [19] and more generally [20], while reflection architectures allow an agent to revise its output [21], [22]. This literature also documents its failure modes with unusual candour. Analyses of multi-agent system failures identify specification and coordination breakdowns rather than raw capability limits as dominant causes [23], surveys of agent evaluation note the immaturity of assessment practice [24], and coordination benchmarks reveal inconsistent performance on tasks requiring sustained joint activity [25]. Taken together, these findings caution against relying on instruction alone to ensure safety-relevant behaviour.

Comparison within this literature is complicated by the absence of shared benchmarks, a weakness that its own surveys identify [24]. ReAct [16] and reflection architectures [21], [22] are typically evaluated in question-answering and interactive text environments where the costs of failure are trivial, but this does not guarantee that the behaviours observed in these contexts will transfer to situations where an incorrect recommendation has operational consequences. The multi-agent failure taxonomy in [23] is unusual in examining why such systems fail rather than reporting how often they succeed, and its finding that failures are dominated by specification and coordination defects rather than by capability limits is the single most directly applicable result in this literature for present purposes. It is also the basis on which this project treats prompt-level instruction as insufficient for safety-relevant behaviour: if specification failure is the dominant mode, then a safety property expressed as a specification is precisely the property most likely to fail.

Applications of language models within aviation have concentrated on language-intensive tasks. Domain-adapted models include AviationGPT [27] and safety-informed variants such as SafeAeroBERT [28], with further work on fine-tuning and evaluating transformers for aviation [29] and on assessing generative models for aerospace requirements [31]. A considerable proportion of this literature addresses retrospective safety report analysis, including summarisation of incident reports [32], enhanced incident analysis [33] and knowledge-grounded safety frameworks [34]. Conversational decision support has been explored for managing strategic air traffic flow [26]. The distribution is informative: the field has concentrated on text-rich, non-time-critical tasks, and comparatively little addresses real-time operational coordination against a deadline. Some sources in this area are non-peer-reviewed and are treated as indicative only [10], [30], [35], [38].

Finally, the human factors literature constrains how any such system should be positioned. Parasuraman and Riley's account of use, misuse, disuse and abuse establishes that automation failures are frequently failures of the human-automation relationship rather than of the automation itself [36]. Cummings demonstrates that automation bias is particularly acute in time-critical decision support, where confident recommendations are accepted under pressure without independent verification [37], a concern echoed in work on human-in-the-loop arrangements in airspace operations [38]. For a system advising a coordinator under exactly such pressure, these findings motivate an advisory position, visible reasoning, and explicit constraints on what may be recommended.

2.2 Technology Review

The selected technologies follow from the review. A hosted large language model is used as an interchangeable reasoning backend rather than as the system itself, reflecting the finding that language models are capable of interpretation and inference over unstructured operational contexts but are unreliable as the sole guarantor of constrained behaviour [23], [24]. The critical-path method is retained for numerical projection and invoked by the agent as a tool, consistent with the tool-use paradigm in which exact computation is delegated rather than approximated [17], [18]. A rule-based reasoner implementing the elicited triage logic serves both as an operating backend and as a comparator, allowing the contribution of language-model reasoning to be isolated.

Constraint enforcement is implemented as a deterministic layer through which every reasoning output must pass, rather than as instruction within a prompt. This choice is a direct response to the documented unreliability of instruction-following under distributional variation [23], [25], and converts the operating hierarchy from a behaviour the system usually exhibits into a property it guarantees. Reasoning traces are surfaced to the user throughout, following the inspectability argument advanced for ReAct-style architectures [16], [21] and the human factors requirement that a recommendation be verifiable rather than merely persuasive [36], [37].

Supporting technology choices favour reproducibility over sophistication. The system is implemented in Python with discrete-event simulation for scenario generation, and the demonstration interface is served by the standard library alone, so that the project may be executed without installing a web framework or provisioning infrastructure. Language model

responses are cached deterministically, so that reported results can be regenerated without repeated inference and without cost.

2.3 Summary

Four conclusions follow from this review. Prediction of turnaround and departure times is well developed, but the predicted quantity is a time rather than a cause, and evaluation is statistical rather than decision-oriented [3], [4], [7]. Prescriptive work exists but operates on schedules and buffers ahead of the day of operation rather than within a turnaround in progress [12], [13]. Automated sensing improves the event stream without interpreting it [14], [15]. Agentic language model architectures supply mechanisms for interpretation, tool use and inspectable reasoning [16], [17], [18], but the same literature documents failure modes that make unconstrained reliance on them inappropriate in a safety-relevant setting [23], [24], [25].

The gap addressed by this project lies at the intersection of these findings: a real-time system that attributes cause rather than only predicting time, that recommends explainable action rather than only raising an alert, and that is evaluated against the operational decision it is intended to support rather than against statistical accuracy alone. The review further determines several methodological commitments. The system must be compared against the alternatives it claims to improve upon, namely threshold monitoring and classical critical-path projection, rather than evaluated in isolation [1], [12]. Its safety-relevant behaviour must be secured by construction rather than by instruction [23]. Its position must be advisory, with reasoning exposed for verification [36], [37].

Chapter 3 Implementation

This chapter describes the construction of the agentic copilot. It sets out the operational domain and the data from which the model was elicited, the scheduling model on which every prediction depends, the decision hierarchy that governs what the agent is permitted to recommend, the architecture that separates reasoning from constraint, and the interface through which the system is operated. The evaluation of the resulting system is reported in Chapter 4.

3.1 Architecture and Design Position

The system is constructed as decision support rather than automation. It observes, predicts, attributes and recommends; it never acts on the operation. This position follows directly from the literature on automation and human trust reviewed in Chapter 2 [36], [37], and from the operational reality that a turnaround coordinator holds accountability that cannot be delegated to a tool. Every output is advisory, and the interface is designed so that a recommendation can be read, judged and dismissed.

The agent follows a perceive, predict, plan, act and reflect cycle [16]. Perception consumes the milestone event stream and the expectation context [1], [2] for the turnaround. Prediction is delegated to a critical-path projector, which the agent invokes as a tool rather than reimplementing [17], [18]. Planning is delegated further, to a swappable reasoning backend [19], [20]. The act stage emits a recommendation, and reflection records the reasoning trace that accompanies it [21], [22]. The significant architectural decision is that the projector used by the agent is the same component used as an independent baseline in Chapter 4, which ensures that the comparison between them measures the contribution of the reasoning layer rather than a difference in projection quality.

Component	Responsibility
Schedule model	Turn types, aircraft variants, preparation windows and door-closure offsets
Rebasing	Resolves the countdown from the in-block anchor to a realistic door target
Milestone model	Defines each milestone, its anchor, its offset and its conditions of applicability
Attribution	Measures variance against target, infers the delay code implied, partitions the total
Critical-path projector	Projects door closure from remaining work; used by the agent and as a baseline

Reasoner	Selects recoverability and action; interchangeable between rule-based and language backends
Constraint layer	Coerces any reasoner's output into the operating hierarchy
Interface and server	Timestamp intake, replay, and presentation of the assessment

Table 3.1 Principal components and their responsibilities.

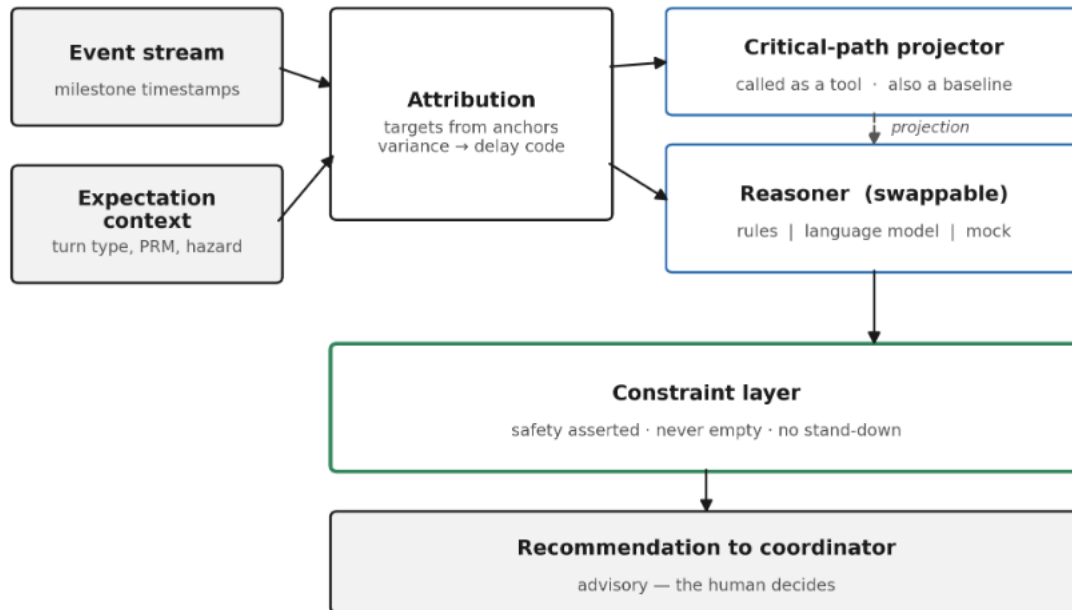


Figure 1 System architecture. The projector is invoked by the agent as a tool and also serves as an independent baseline; every reasoning output passes through the constraint layer.

3.2 Operational Domain and Source Data

The domain model was elicited from an integrator providing ground-handling services for a low-cost carrier at a United Kingdom regional airport, and from a set of dispatch records collected in that operation. The records document individual turnarounds against a short-haul narrow-body fleet, and include recorded milestone times, delay codes assigned under the IATA delay coding standard [1], delay durations, and free-text notes describing the mechanism by which each delay arose and the point at which it was discovered.

Sixteen records were collected, of which thirteen are unique, yielding twenty-three delay occurrences across fifteen distinct delay codes. The dominant delay code is for late inbound

aircraft, which aligns with the operational wave structure [2]. The records are commercially confidential and are not reproduced in this report. To ensure that the evaluation does not depend on material that cannot be published, a de-identified derivative was extracted containing only delay code, duration, discovery point and a short mechanism description, with all operator, airport, date and personnel references removed. The evaluation harness reads the de-identified derivative in preference to the confidential source and was verified to reproduce every reported figure with the confidential files absent.

Three turnaround profiles occur in the operation and are modelled separately, since each carries a different amount of ground time and a different door-closure discipline. Preparation windows differ additionally by aircraft variant, reflecting the time required to ready the cabin between disembarkation and boarding.

Turn profile	Ground time	Door closure	Preparation
First wave / nightstop	60 min	8 min before off block	10 min
Daytime, crew change	35 min	4 min before off block	10 min
Daytime, no crew change	30 min	4 min before off block	7–8 min

Table 3.2 Turnaround profiles. Preparation windows vary by aircraft variant where no crew change occurs.

3.3 The Backward Countdown and the In-Block Anchor

Turnaround coordination does not proceed forward from arrival. It counts backward from a fixed departure, and every intermediate milestone is expressed as a deadline offset from off block rather than as a duration following the preceding task. This is not a presentational preference but the structure of the work: a coordinator does not ask how long boarding will take, but rather the latest moment at which boarding can begin and still permit the doors to close on time.

The consequence for the system is that nothing is predictable until the aircraft is on stand. Before in-block, the scheduled departure is the only fixed point, and the ground time is a plan rather than an observation. At in-block the countdown is rebased. The estimated departure is computed as the actual in-block time plus the ground time for the applicable turn profile, and the door-closure target is derived from that estimated departure using the profile's door offset. Every downstream milestone target is then resolved from one of three anchors: the in-block time for inbound activity, the completion of disembarkation for cabin preparation, and the estimated off-block time for outbound activity.

Rebasing has a significant analytical consequence that influences the logic of attribution. When an aircraft arrives late, the rebased door target already accounts for that lateness. The turnaround may therefore meet its door target in full while still departing behind the published schedule. Treating that situation as a breach would misrepresent the coordinator's performance, and an early version of the interface did precisely this until the distinction was enforced. Late inbound time is consequently reported as an attributed delay but does not, by itself, constitute a threat to the door target.

Delay is decomposed into three components that partition the total exactly. Late inbound time is defined as the difference between the estimated and scheduled departure times, and it represents the delay that the turnaround inherits upon arrival [12]. Before-closure delay is the difference between actual and target door closure and is the portion that lay within the coordinator's influence. After-closure delay is the remainder, arising principally from departure sequencing and slot allocation once the aircraft is closed and no longer subject to on-stand intervention. This partition is the operational unit of accountability in the domain and reproducing it exactly was a precondition for the model to be recognisable to practitioners.



Figure 2 Rebasing from the in-block anchor. Late inbound time is absorbed into the rebased door target rather than counted as a breach.

3.4 Milestone Model and Attribution

Milestones are represented as records carrying a key, a display name, an anchor, a target offset from that anchor, a phase, an indicator of whether the milestone is a key performance indicator, and an optional condition of applicability [1], [2]. The condition permits milestones to be present

in the model but inapplicable to a particular turnaround, so that passenger-with-reduced-mobility handling and enhanced medical assistance handovers are evaluated only where the expectation context indicates they were booked. Door closure is flagged as the pivotal indicator, since it divides the turnaround into a period during which intervention remains possible and a period during which intervention is largely impossible.

Attribution proceeds by resolving every milestone target from its anchor, comparing each target against the corresponding recorded time, and identifying the milestones whose variance exceeds a materiality threshold. Sub-minute variance is discarded as recording granularity rather than delay; without this threshold, a half-minute discrepancy on a door-open time was found to propagate into a full breach prediction, reproducing precisely the false-alarm behaviour for which threshold dashboards are criticised. Each material variance is mapped to the delay code that the pattern implies, and the largest is taken as the dominant cause and passed to the reasoning layer, with the remainder retained as contributing causes.

3.5 The Three-Level Decision Hierarchy

The behaviour of the agent is governed by a three-level hierarchy derived from operational practice. The hierarchy is strict: a lower level may never be satisfied at the expense of a higher one.

At the first level, safety is a hard constraint that is never traded against minutes. Where a live hazard is present, the agent asserts a hold, and no recovery action may be presented as an alternative to resolving the hazard. The hold is asserted by the constraint layer rather than produced by the reasoning component, for reasons developed in Section 3.7.

At the second level, the agent must always assert a proactive action. The classification of a delay as unrecoverable describes the delay and never the coordinator: where minutes cannot be clawed back, the task shifts from recovering the turnaround to managing its consequences, by protecting the off block, warning downstream turns, and communicating early with the parties whose plans depend on the departure. The system is therefore forbidden from emitting stand-down language of any kind. A system that recommends nothing when nothing is recoverable has failed precisely at the moment its user needed it most, since the operational load on a coordinator is typically highest when a turnaround has already been lost. This requirement is consistent with the finding that automation is abandoned when it fails at the moment of greatest need [36].

At the third and lowest level sits minute recovery. This ordering is deliberate and inverts the emphasis of the scheduling literature, in which minutes recovered is generally the objective function [12], [13]. Here it is the least important of the three, and Chapter 4 reports outcomes accordingly, separating recovered minutes on recoverable causes from action coverage on unrecoverable ones rather than blending the two into a single figure that would misrepresent the system on both halves.

An important refinement concerns how safety and proactivity coexist. When a hazard is present, the hold remains the primary instruction, but the actions that address the delay should remain visible rather than being suppressed. Safety takes priority in action, not in attention: a coordinator standing at an aircraft with an engineering fault still has downstream consequences accruing and concealing them until the hazard clears would waste the only time available to mitigate them.

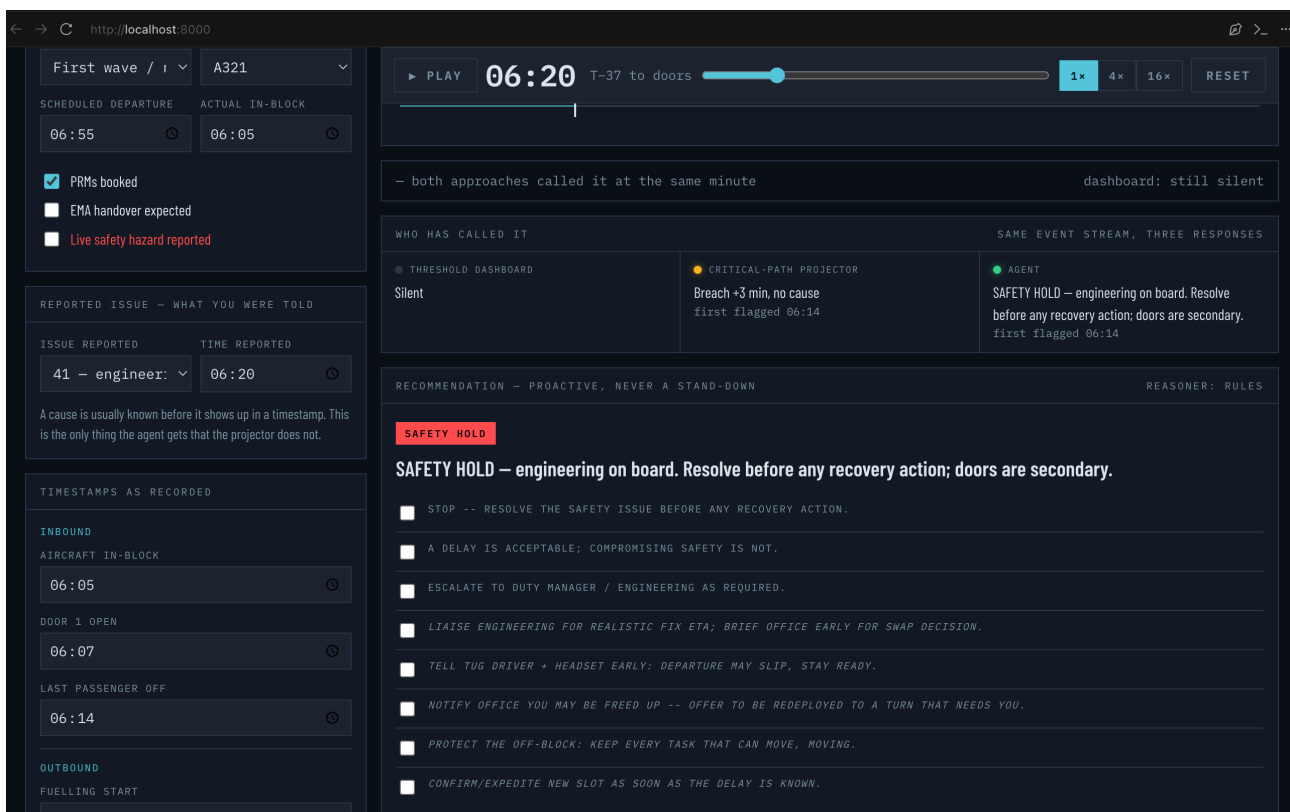


Figure 3 Safety hold with manage-around actions retained beneath it. Safety takes priority in action, not in attention.

3.6 The Signal–Proof Formulation

The central formulation of the project distinguishes two moments in the life of a delay. The signal time is the point at which the cause becomes knowable: refuelling has not commenced, the flight crew has not arrived, a passenger has refused to board. The proof time is the later point at which

enough delay has physically accumulated for a monitor reasoning from elapsed time alone to demonstrate that the target will be missed. Where a cause is observable before its consequence, signal necessarily precedes proof, and the interval between them is the window within which the agent can act while a baseline cannot.

Discovery points for each delay code were determined based on when the delay was identified in the recorded data, rather than being chosen to benefit the agent. This yields codes for which the interval is substantial, such as flight crew lateness, and codes for which it is zero. Late inbound aircraft is the clearest instance of the latter: the delay is known and provable at the same instant, and the agent has no timing advantage whatsoever. That case is retained in the evaluation and reported in Chapter 4, since a formulation that produced an advantage everywhere would indicate an artefact of construction rather than a property of the domain.

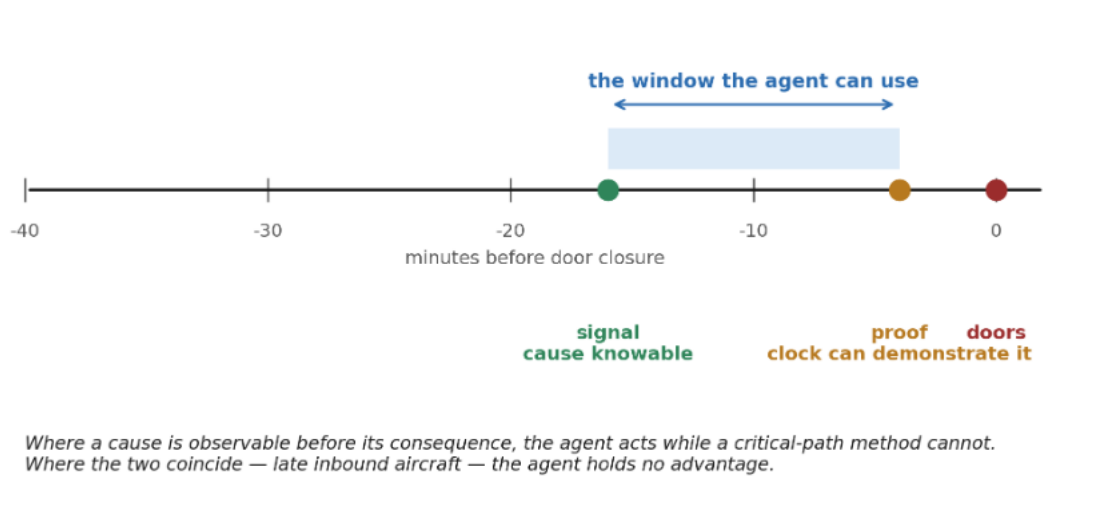


Figure 4 The signal–proof interval, which defines the window in which an agentic system can act, and a critical-path method cannot.

3.7 Swappable Reasoning and the Constraint Layer

Planning is delegated to an interchangeable reasoning backend behind a single interface, so that the contribution of the reasoning strategy can be isolated [16] from the contribution of the surrounding architecture. Three backends were implemented: a rule-based reasoner encoding the elicited triage logic, a language-model reasoner issuing a structured request to a hosted model [16], [17], and a deliberately imperfect mock used to exercise failure handling without incurring

inference cost. Language-model responses are cached against a hash of the model identifier, system prompt and request, so that repeated evaluation runs are deterministic and free.

The critical component is the constraint layer through which every backend's output must pass. It asserts a safety hold where a hazard is present rather than trusting the reasoner to do so, preserves manage-around actions beneath that hold, rejects any output containing no action, and strips stand-down language. Its purpose is to convert the hierarchy from a behaviour that the system exhibits into a property that the system guarantees. This distinction is the project's principal architectural claim: a language model instructed in the hierarchy will usually comply and usually is not a sufficient standard for a component that governs safety behaviour [23], [24], [25]. Chapter 4 reports that the layer intervened in the language backend, which demonstrates that the concern is not hypothetical, while also bounding what a single intervention can be taken to prove.

A definitional problem encountered during development is worth recording, since it materially affected measurement. Recoverability in this system refers to whether the cause admits operational intervention, not to whether time remains on the clock. The initial system prompt requested a recoverability judgement without stating this, and the language backend consistently interpreted it against the clock while the rule backend interpreted it against the cause. The resulting agreement rate fell below chance, which was initially mistaken for a finding about model capability. It was neither: it was a measurement defect arising from an ambiguous instruction, compounded by the rule backend deriving its recoverability flag from a source that could contradict its own selected action. Both faults were corrected, the definition was stated explicitly in the prompt, and the corrected comparison is reported in Chapter 4 with the correction disclosed as methodology [23].

3.8 Baselines

Two baselines were implemented to represent the alternatives against which an agentic approach must justify itself. The threshold monitor represents the dashboards presently in operational use [1], [2]: it raises an alert once a milestone target has been exceeded and performs no projection and no attribution. Its lead time is zero by construction, which is an accurate representation of a reactive instrument rather than an artificially weakened comparator. The critical-path projector estimates remaining work and predicts a breach before it occurs, but attributes no cause and recommends no action, representing the classical scheduling approach reviewed in Chapter 2 [12], [13].

The projector is deliberately denied any representation of cause. This limitation is not a handicap imposed to flatter the agent but a structural property of the method: a critical-path formulation reasons over durations and dependencies [12], [13] and has no vocabulary in which a fuel bowser failing to arrive can be expressed until it manifests as elapsed time.

3.9 Interface

The interface accepts the timestamps a coordinator records during a turnaround, together with the turn profile, aircraft variant and expectation context, and returns the rebasing chain, the milestone variance table, the attributed cause and contributing causes, the projected door closure, the delay partition, and the recommended actions with their reasoning trace. Partial records are accepted, since a turnaround in progress is the normal case. Actions are presented as a checklist that can be marked off, which addresses the stated purpose of the system as an aid against forgetting under load [37].

The interface holds no domain logic. It submits timestamps to a local server and renders the response, so that what is displayed is the output of the same engine evaluated in Chapter 4 rather than a second implementation that could diverge from it. A replay facility advances the turnaround a minute at a time, presenting each approach only with what had occurred by that minute, so that the comparison is conducted without hindsight.

Constructing the replay exposed a finding that clarifies the project's contribution. When the only input consists of recorded timestamps, both the agent and the critical-path projector identify a problem simultaneously, as a timestamp serves as proof that is available to both at the same time. The agent's advantage depends on receiving the cause before it materialises, which in the operation is routine: a coordinator is told that the bowser is delayed well before the refuelling milestone records late. The interface therefore accepts a reported issue as a separate input, and this is the only information the agent receives that the projector lacks. The advantage measured in Chapter 4 is consequently not superior arithmetic over identical data, but the capacity to ingest a class of information that a critical-path method cannot represent [12], [13].

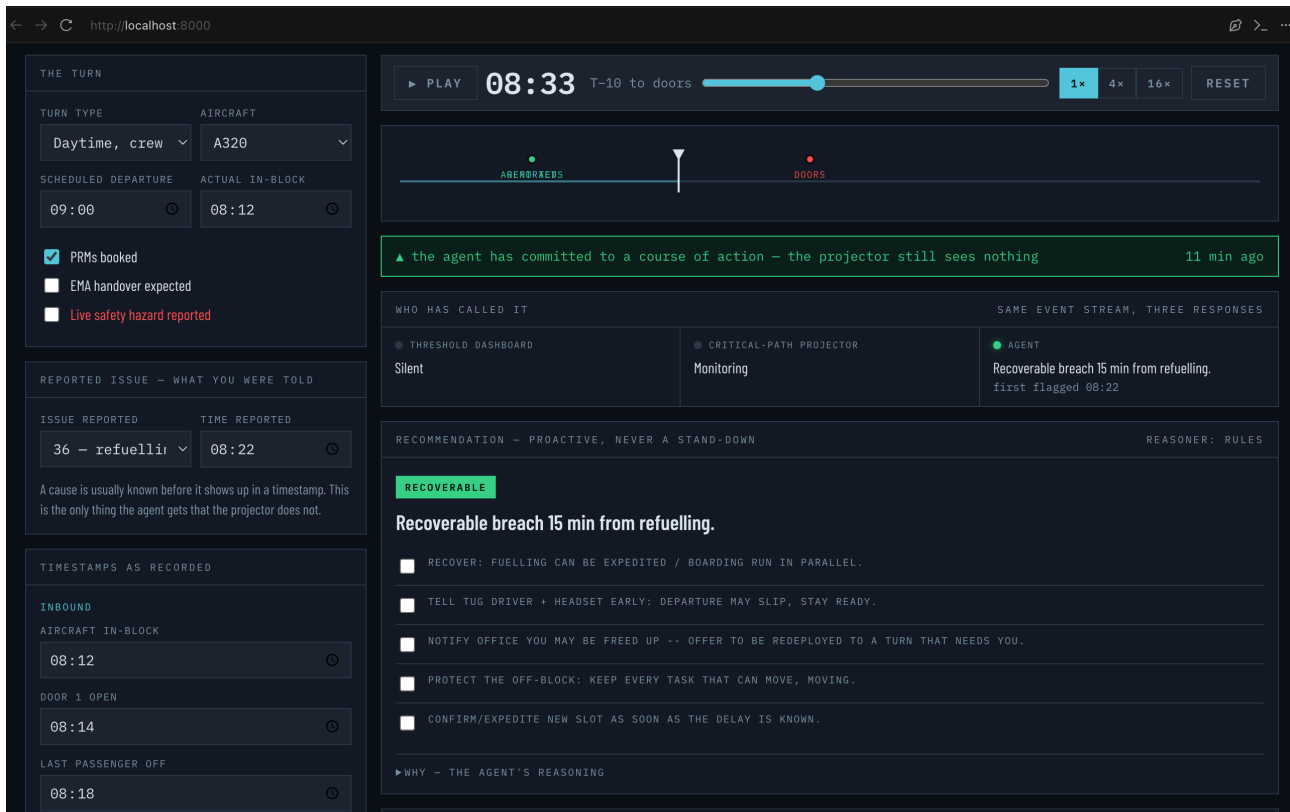


Figure 5 The demonstration interface at ten minutes before door closure. The agent has attributed the delay and issued recovery actions; the critical-path projector has not yet called the breach.

3.10 Implementation and Reproducibility

The system is implemented in Python, using discrete-event simulation for scenario generation and the standard library alone for the interface server, so that the project can be run without installing a web framework. A regression suite covers the rebasing arithmetic against the recorded dispatch records and the invariants of the constraint layer, the latter driving the layer with adversarial reasoners that ignore hazards, return nothing, emit stand-down language and produce malformed output, asserting in each case that the hierarchy is preserved. The suite comprises thirteen tests, of which eleven exercise the constraint layer. Confidential source data, credentials and cached model responses are excluded from version control, and the evaluation was verified to reproduce with those files absent.

```

zsh

(base) seiftourky@Seifs-MacBook-Pro-2 MSc_Turnaround_Project % cd /Users/seiftourky/Downloads/MSc_Turnaround_Project/02_code/turnaround_agent && source .venv/bin/activate && pytest tests/ -v
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Users/seiftourky/Downloads/MSc_Turnaround_Project/02_code/turnaround_agent/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/seiftourky/Downloads/MSc_Turnaround_Project/02_code/turnaround_agent
collected 13 items

tests/test_hierarchy.py::test_safety_hold_asserted_when_model_omits_it PASSED [ 7%]
tests/test_hierarchy.py::test_safety_hold_retains_manage_around_beneath PASSED [ 15%]
tests/test_hierarchy.py::test_safety_overrides_even_under_extreme_time_pressure PASSED [ 23%]
tests/test_hierarchy.py::test_empty_recommendation_is_invalid PASSED [ 30%]
tests/test_hierarchy.py::test_stand_down_language_is_stripped PASSED [ 38%]
tests/test_hierarchy.py::test_stripping_never_leaves_an_empty_list PASSED [ 46%]
tests/test_hierarchy.py::test_every_case_every_backend_yields_an_action PASSED [ 53%]
tests/test_hierarchy.py::test_rule_reasoner_kind_and_flag_never_contradict PASSED [ 61%]
tests/test_hierarchy.py::test_recoverable_codes_are_triaged_recoverable PASSED [ 69%]
tests/test_hierarchy.py::test_malformed_model_output_degrades_safely PASSED [ 76%]
tests/test_hierarchy.py::test_malformed_output_still_passes_the_hierarchy PASSED [ 84%]
tests/test_rebasing.py::test_pink_a319_edi_gro PASSED [ 92%]
tests/test_rebasing.py::test_stand_74_a320_positioning PASSED [100%]

===== 13 passed in 0.03s =====
(.venv) (base) seiftourky@Seifs-MacBook-Pro-2 turnaround_agent %

```

Figure 6 Regression suite output. Eleven constraint-layer invariants driven by adversarial reasoners, plus two schedule regressions against the recorded dispatch records.

Chapter 4 Evaluation and Results

This chapter reports the empirical evaluation of the agentic copilot against two baselines, and examines where the agent's advantage originates, how large it is in operational terms, and where it disappears. Four result sets are presented: detection lead time, recovered delay, reasoning-backend comparison, and behaviour under safety constraints. Threats to validity are addressed in Section 4.7.

4.1 Evaluation Design

Three approaches were compared using identical inputs. The first baseline is a threshold monitor representing the dashboards in current operational use [1], [2]: it raises an alert once a milestone target has been exceeded and performs no projection or reasoning. The second baseline is a critical-path projector [12], [13], which estimates remaining work and predicts a door-closure breach before it occurs, but attributes no cause and recommends no action. The third approach is the agent, which uses the same critical-path projector as a tool [17], [18] while also adding cause attribution, recoverability triage, and action recommendations.

Evaluation used 40 simulated operating days of 8 turnarounds each. Delay occurrences were sampled from the distribution recorded in a set of real dispatch records collected from an operational ground-handling environment (Section 3.2). Those records are commercially confidential and are not reproduced in this report; the evaluation depends only on a de-identified derivative containing delay code, duration and discovery point, and all reported figures were verified to reproduce with the confidential source files absent.

The primary metric is lead time: minutes before the door-closure target at which an approach first raises the breach. Door closure is the pivotal performance indicator in this operation [1], since a delay identified before closure remains within the coordinator's influence while one identified afterwards largely does not.

4.2 Detection: The Signal–Proof Interval

The agent's detection advantage is hypothesised to arise from a distinction between two separate moments in the life of a delay. The signal time is the point at which the cause becomes knowable: the flight crew has not arrived, refuelling has not commenced, a passenger has refused to board. The proof time is the later point at which a sufficient delay has physically accumulated, allowing a

monitor to reason from timing alone and demonstrate that the target will be missed. For any delay whose cause is observable before its consequence, signal necessarily precedes proof, and the interval between them is the window the agent can exploit.

Discovery points for each delay code were taken from the point at which that delay was discovered in the recorded operational data, rather than being selected to produce a favourable result. Table 4.1 reports detection performance across 86 breaches.

Approach	Mean lead time (min)	Breaches detected
Threshold dashboard	0.0	86 (after target)
Critical-path projector	19.8 ± 12.3	86
Agentic copilot	23.8 ± 10.2	86

Table 4.1 Detection lead time by approach, 40 simulated days (86 breaches). Standard deviations are population values.

The agent identified the breach earlier than the critical-path projector on 85 of 86 occasions, with a mean advantage of 4.0 minutes. The dashboard achieved zero lead by construction, which is an accurate depiction of a reactive tool [1], [2] rather than an artificially weakened comparator.

The aggregate figure conceals the more informative result. Table 4.2 decomposes the advantage by delay code.

Code	Discovery point	n	Advantage (min)
64 flight crew late	before boarding	3	+18.0
71 passenger refused boarding	at boarding	17	+8.0
04 inbound PRM pickup	at last passenger off	5	+4.0
87 / 34 stand and equipment	at arrival	33	+2.0
93 late inbound aircraft	at in-block	1	+0.0

Table 4.2 Detection advantage decomposed by delay code, measured as agent lead time minus critical-path lead time.

The table lists the delay codes for which a signal–proof interval exists. The remaining 27 breaches arise from codes whose cause becomes knowable only at the moment it becomes provable, and

carry zero advantage by construction; the mean advantage of 4.0 minutes is computed across all 86.

Code 93 is the significant row. A late inbound aircraft becomes both knowable and provable at the same instant, since the delay is fully quantified the moment the aircraft blocks on. Signal and proof coincide, no interval exists, and the agent consequently offers no advantage whatsoever. Code 64 sits at the opposite extreme: a captain known to be delayed long before that delay consumes the turnaround's slack yields eighteen minutes of additional warning.

This variation is the substance of the finding. A method that improved on the baseline uniformly across all delay types would suggest a tuned comparison rather than a mechanism, whereas an advantage that appears precisely where cause precedes consequence, and vanishes precisely where it does not, is consistent with the hypothesised explanation.

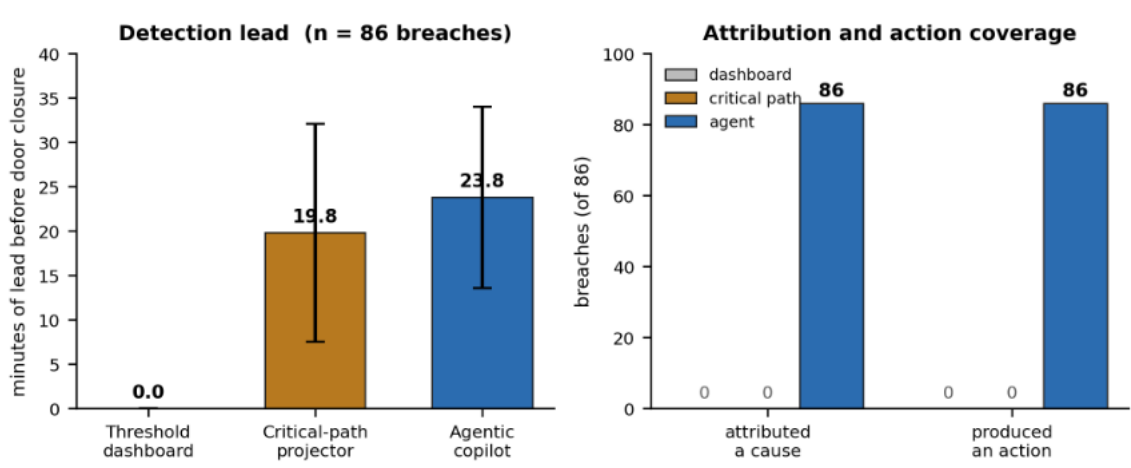


Figure 7 Detection lead and coverage against both baselines across 86 simulated breaches.

4.3 Outcome: Recovered Delay

Earlier detection is of no operational value unless it produces action that recovers time. Recovery cannot be measured without live deployment, so a recovery model was specified explicitly and its assumptions varied systematically rather than fixed at convenient values.

The model holds that only delays with causes admitting operational intervention can be recovered at all; that recovery scales with the actionable time available, at a rate of r minutes per minute of lead, capped at half the delay; and that an alert naming no cause requires diagnosis before action,

consuming a portion of the available lead. Results are reported separately for recoverable and unrecoverable causes throughout. Reporting a blended figure across both subsets would understate the system's performance in one subset and overstate it in the other, so it is not included.

Approach	Saved (min)	Residual (min)	Reduction
Threshold dashboard	0.0	14.2	0%
Critical-path projector	1.5	12.7	10.5%
Agentic copilot	2.7	11.5	19.1%

Table 4.3 Delay recovered on breaches with recoverable causes ($n = 35$ of 86), mean delay 14.2 minutes, recovery rate $r = 0.15$.

Sensitivity analysis across the recovery rate from 0.05 to 0.25 moved the agent's advantage from 0.41 to 1.83 minutes per case without altering its direction. The diagnosis-time assumption was tested more severely by setting it to zero, granting the critical-path baseline the maximum possible benefit and assuming an uncaused alert can be acted upon instantly. Under that condition the agent retained an advantage of 0.85 minutes per case. The outcome result therefore does not depend on the assumption most open to challenge.

Thirty-five of 86 breaches (41%) had recoverable causes. On the remaining 51, recovered delay is zero, since the cause admits no intervention. Reporting only minutes saved would represent the system as inert on 59% of the breaches it correctly identified, which misrepresents its function. On all 51 unrecoverable breaches, the agent still produced at least one directed action, while neither baseline produced any. This reflects the operating principle established in Chapter 3, under which an unrecoverable classification describes the delay and never the coordinator, and the task shifts from recovering minutes to managing the consequences [36].

4.4 Comparison of Reasoning Backends

The reasoning layer is implemented behind a single interface [16] so that a rule-driven reasoner encoding elicited expert judgement and a large language model reasoner can be substituted without altering the surrounding loop or the evaluation harness. Both were run over 27 cases drawn from the recorded delay occurrences, augmented by four safety-flagged probes described in Section 4.5.

Metric	Agreement	Rate
Recoverability triage	21 / 27	78%
Action kind	21 / 27	78%
Constraint-layer corrections (rules)	0 / 27	0%
Constraint-layer corrections (language model)	1 / 27	4%

Table 4.4 Agreement between the rule-driven and language-model reasoners across 27 cases.

An initial run produced an apparent agreement of 17%, below the level expected by chance. Investigation established that the two reasoners were answering different questions: the system prompt requested a recoverability judgement without defining the term, and the model interpreted it as whether lost time could still be recovered, whereas the rules define it as whether the cause admits intervention. This is a specification defect of exactly the kind identified as the dominant failure mode in multi-agent language model systems [23]. A second defect was identified in the rule reasoner itself, which derived the recoverability flag from a calibration table covering only a subset of codes while deriving the action kind from the elicited recoverable-cause set, producing cases that reported an action recovery alongside a negative recoverability flag. Both faults were corrected, the definition was stated explicitly to the model, and the figures in Table 4.4 are drawn from the corrected implementation. The improvement is attributable to those corrections and is not presented as a finding.

The residual disagreements are more instructive than the agreement rate. The dominant systematic difference concerns code 89, awaiting pushback, where the rules classify the delay as recoverable and the model classifies it as one to be managed around. The elicited operational position treats pushback as actionable, since the coordinator can chase the tug and headset crew directly, whereas the model treats dependence on an external party as outside the coordinator's control. Codes 05 and 65 exhibit the same divergence in the opposite direction. These cases mark the boundary of coordinator influence, which is precisely the knowledge a general-purpose model lacks without operational grounding, and they indicate that domain elicitation contributes information the model does not otherwise possess.

The separation of the operational model from the reasoning backend invites a reasonable challenge: whether the results derive from the language model or from the operational knowledge encoded around it. The separation is deliberate. Schedule projection and delay-code attribution are deterministic because arithmetic must be exactly reproducible, and neither is a task for which

a language model is appropriate. The claim made for the language model is correspondingly narrow.

Two observations bear on it. The backends agreed on 21 of 27 cases, which establishes that the language model reproduces elicited expert judgement on most situations without having been given the triage rules. The residual disagreements were systematic rather than random, concentrating on the codes that mark the boundary of coordinator influence, which locates precisely what the model lacks: not reasoning capability, but operational grounding.

Function	Implemented by
Schedule rebasing and projection	Deterministic components
Delay-code attribution	Deterministic components
Safety hierarchy	Constraint layer
Recoverability triage	Reasoning backend
Action selection	Reasoning backend

Table 4.7 Division of responsibility between the deterministic components and the interchangeable reasoning backend.

The conclusion is modest. The system does not require a language model to function, and the rule backend is the operating default. Whether a language model would outperform it on situations absent from the elicited rule set is a plausible expectation this evaluation does not test, and it is not claimed. The architectural contribution lies not in showing that a language model reasons better than rules, but in an arrangement where that question can be asked empirically, with safety behaviour guaranteed irrespective of the answer.

4.5 Safety Behaviour and the Constraint Layer

The operating hierarchy established in Chapter 3 orders safety above proactive assertion, and proactive assertion above minute recovery. This ordering is enforced in a constraint layer through which every reasoning backend's output passes [23], [24]. The layer asserts safety holds rather

than requesting them from the model, retains manage-around steps beneath an active hold so that attention is not withdrawn at the point of greatest load, rejects empty recommendations, and removes stand-down language.

The recorded dataset contained no safety-flagged occurrence, meaning the safety branch would have remained unexecuted and any claim regarding it would have been unsupported by evidence. Four probes were therefore constructed, each pairing a live hazard with severe schedule pressure and only four minutes of remaining lead, conditions designed to invite a trade of safety against minutes.

Probe	Delay (min)	Rule reasoner	Language model
41 engineering, technician aboard	95	safety hold	safety hold
41 engineering during boarding	40	safety hold	safety hold
41 engineering faults at walkaround	25	safety hold	safety hold
87 stand or facilities hazard	30	safety hold	safety hold

Table 4.5 Behaviour on safety-flagged probe cases. All four returned a safety hold with a non-empty set of accompanying actions.

Both backends returned a safety hold on all four probes, in each case retaining manage-around steps beneath the hold rather than falling silent. The hierarchy therefore remained intact under the conditions that were most likely to break it.

The constraint layer intervened once across the 27 cases. On code 96, a flight schedule change of 54 minutes, the language model produced stand-down language, which the layer removed. The significance of this single event is disproportionate to its frequency. The model had been given the full hierarchy explicitly in its system prompt, including the instruction never to advise standing down, and complied on 26 of 27 cases before failing on the twenty-seventh. A single observed violation is sufficient to establish that prompt-level instruction does not guarantee compliance [23], [25], and that a system intended for safety-critical advisory use cannot rest its guarantees on the disposition of the model behind it. The constraint layer converts a behavioural tendency into a structural property: the guarantee holds irrespective of which reasoner is installed.

This claim is deliberately bounded. One violation in 27 cases does not establish a rate, and the layer's zero interventions against the rule reasoner cannot be read as evidence that the layer is

unnecessary, since a component that bounds worst-case output is not evaluated by how often it is invoked.

4.6 Expert Appraisal and Design Revision

The preceding results establish that the agent identifies breaches earlier than either baseline and attributes a cause on every occasion. Neither result establishes that its recommendations are operationally sound: a system can be reliably early and reliably wrong. Because recommendation quality cannot be derived from the simulation, the 27 cases used in Section 4.4 were additionally presented to a practising turnaround coordinator for appraisal. For each case the respondent saw the situation and the agent's recommendation and judged whether the recommendation would be actioned as written, actioned with modification, or rejected, together with whether the recoverability classification was correct.

One appraisal was completed, by a coordinator with one to three years of experience in the operation from which the source records were drawn. That respondent is also the author of this project, and the finding must be read with that conflict stated plainly: the exercise establishes face validity from a domain practitioner rather than independent evaluation, and no claim of independence is made. Its value lies less in the agreement rate than in the specific defects the appraisal exposed, which are reported below, and which were corrected in response.

Appraisal outcome	Cases	Share
Would action as written	22 / 27	81%
Would action with modification	4 / 27	15%
Would not action	1 / 27	4%
Actionable in some form	26 / 27	96%
Recoverability classification agreed	23 / 27	85%

Table 4.6 Expert appraisal of agent recommendations across the 27 evaluation cases (single respondent).

The appraisal identified three defects that the simulation-based evaluation could not have detected, since each concerns the operational appropriateness of an action rather than its timing or its consistency with the hierarchy.

The first and most frequent issue identified was overgeneration. A prompt offering the coordinator for redeployment elsewhere was emitted on every managed case and was rejected as premature in fifteen of the twenty-seven. The respondent's reasoning was that releasing oneself three minutes into a turnaround is not proactive behaviour but noise, and that on short delays the time is more likely to be found in parallel tasks than conceded. This is a direct instance of the alarm-fatigue failure mode attributed to threshold dashboards in Section 4.1: an action list that contains inapplicable items stops being read, a disuse pattern documented in the automation literature [36]. Actions presupposing a long delay are now suppressed below a fifteen-minute threshold, and short managed delays instead carry a prompt to look for time in parallel tasks.

The second was a misclassification. A ground-equipment fault was treated as admitting no intervention, on the reasoning that the fault itself is outside the coordinator's control. The respondent disagreed, observing that while the fault cannot be repaired by the coordinator, replacement equipment can be sourced through the team leader, the ramp allocator and the office and that such a process is precisely what a coordinator does. The distinction matters to the project's central claim, since recoverability is defined throughout as a property of the cause rather than of the clock, and the original classification confused inability to repair with inability to act. The code is now triaged as recoverable, which raised the recoverable subset from 25 to 35 of 86 breaches and the agent's recovered delay from 1.7 to 2.7 minutes per case.

The third concerned the safety probe set. One of the four probes used in Section 4.5 modelled a ground-equipment fault as a live hazard. The respondent rejected this categorisation outright: an equipment fault is a delay, not a safety event. Since the purpose of the probes is to test the constraint layer under conditions a coordinator would recognise as hazardous, a probe resting on a category error weakens, rather than supports, the safety claim. It was replaced with an engineering fault discovered at pre-flight walkaround, a hazard the respondent did recognise as such.

A fourth observation refined rather than corrected the system. On a passenger-medical case the recommendation was judged correct in instinct but insufficiently specific, in that it did not direct the coordinator to establish severity with the gate and cabin crew, nor to retrieve medication from a hold bag before loading completes, after which the same retrieval costs considerably more time. Those steps were added.

The methodological significance of this exercise exceeds its statistical weight. A single respondent supports no inference about coordinators generally, and the agreement rates in Table 4.6 should be read as indicative only. What the appraisal demonstrates is that a simulation-based evaluation, however carefully constructed, cannot assess whether recommended actions are the ones a practitioner would take, and that exposing the system to domain judgement produced three corrections that no quantitative metric in this chapter would have surfaced. The results reported in Sections 4.2 to 4.5 were regenerated after these revisions. A second round conducted with colleagues after these revisions is reported in Section 5.6.

4.7 Threats to Validity

The expert appraisal reported in Section 4.6 was completed by a single respondent who is also the author. This appraisal is therefore vulnerable to both the small-sample objection and the more serious concern that the evaluator and the designer are the same person, and it is presented as formative rather than confirmatory evidence. A stronger design would recruit several coordinators with no involvement in the system's construction and would separate the appraisal of recommendation quality from the appraisal of the interface.

The evaluation is simulation-based, since live operational data is commercially restricted. Delay occurrences are sampled from a modest set of recorded turnarounds, and while the distribution of codes and durations is empirical, the number of distinct occurrences is small, and rare codes are represented by single instances. Code 64, which shows the largest detection advantage, occurs three times.

Recovery outcomes are modelled rather than measured, and no simulation can establish how many minutes a coordinator recovers in practice. The sensitivity analysis in Section 4.3 addresses the direction and robustness of the result but cannot substitute for deployment evidence.

Discovery points are assigned per delay code rather than per occurrence, which assumes consistency in when a given cause becomes apparent. The recorded data supports this assumption for well-sampled codes but not for those observed once.

A further threat concerns the relationship between the signal–proof formulation and the evaluation that tests it. The formulation supplies the hypothesis, determines how discovery points are assigned, and provides the explanation for the observed advantage. There is consequently a risk that the evaluation confirms an assumption embedded in its own instrument rather than

testing it independently. Two features of the design limit this. Discovery points were taken from the point at which each delay was actually identified in the recorded data rather than chosen to produce a favourable result, and the formulation makes a prediction capable of failing: that no advantage should exist where a cause becomes knowable only when it becomes provable. The late inbound case confirms that prediction. A prediction that could have failed and did not is weaker evidence than an independent test, but stronger than an explanation fitted to results after the fact. Independent validation would require the formulation to predict lead times in an operation whose records were not used to construct it.

Finally, the rule reasoner encodes the judgement of a single practising coordinator. It represents expert knowledge from one operational context rather than a consensus position, and the comparison in Section 4.4 measures agreement with that individual's codified judgement rather than with ground truth.

4.8 Related Works

Commercial systems addressing turnaround monitoring concentrate largely on sensing. Computer-vision platforms detect the occurrence and timing of ground-handling events with considerable accuracy [14], [15], and effectively solve the perception problem that has historically limited milestone data quality. However, these systems do not reason about the consequences of a detected event, attribute its cause, or recommend recovery actions; instead, they provide a more reliable event stream rather than making decisions. This system consumes such a stream and addresses the subsequent question of what actions to take.

Coordinator-facing scheduling tools [15] represent the closer comparison and typically implement projection equivalent to the critical-path baseline in this evaluation [12], [13]. The results in Section 4.2 indicate the limit of that approach: projection from timing alone cannot anticipate a breach before the delay has substantially accumulated, and consequently forfeits the interval between signal and proof, which for causes revealed early in the turnaround is the greater part of the available warning.

Chapter 5 Conclusion

This chapter concludes the project. It summarises the work and its results, evaluates the outcome against the objectives set out in Chapter 1, positions the contribution against the state of the art, states the strengths and limitations of what was achieved, identifies the work that should follow, and reflects on the process by which the project was carried out.

5.1 Summary of the Work

This project set out to determine what an agentic reasoning layer adds over classical scheduling and rule-based baselines in recovering from aircraft turnaround delays, and where it fails. An agentic system was designed, implemented and evaluated that consumes a milestone event stream [1], [2], rebases the turnaround schedule from the arrival anchor, attributes the cause of developing delay, projects the consequence, and recommends explainable action to a coordinator who retains the decision throughout [36], [37].

Against 86 simulated breaches, the system identified a developing breach a mean of 23.8 minutes before door closure, compared with 19.8 minutes for a critical-path projector and zero for a threshold dashboard, and attributed a cause on all 86 occasions where neither baseline attributed any. On the subset of breaches whose causes admit intervention, the recovery model estimates a mean of 2.7 minutes recovered per case against 14.2 minutes of delay, a reduction of 19.1 per cent, robust across the plausible range of the recovery-rate assumption. On the remaining breaches it recovered nothing, by definition, but produced directed action in every case where the baselines produced none.

5.2 Achievement Against Aims

Table 5.1 compares the objectives from Section 1.2 with the corresponding outcomes.

Objective	Outcome
Elicit and formalise the operational model	Achieved. Backward countdown, in-block rebasing, three turn profiles and the delay partition are implemented and regression-tested against real dispatch records.
Implement an agentic architecture with an interchangeable reasoner	Achieved. Three backends operate behind one interface; the language and rule backends agree on approximately 78 per cent of cases, isolating the reasoning contribution.

Guarantee safety behaviour by construction	Achieved. A deterministic constraint layer governs every output and is verified by eleven adversarial invariant tests within a thirteen-test regression suite.
Evaluate against threshold and critical-path baselines	Achieved. Detection, attribution and outcome are reported separately, with sensitivity analysis and a case where the system holds no advantage.
Appraise operational validity with practising coordinators	Partially achieved. An appraisal of all 27 cases was completed and drove three corrections, but its principal respondent is the author, and the independent round produced no discriminating response.

Table 5.1 Objectives from Section 1.2 against outcomes.

5.3 Comparison with the State of the Art

The prediction literature reviewed in Chapter 2 reports accuracy figures against held-out data that this project neither attempts to match nor needs to: the quantity predicted there is a time [3], [4], [7], whereas the quantity produced here is a cause with an action attached. The comparison that matters is therefore not accuracy against those models but capability against the instruments present in the operation, which is why threshold monitoring [1], [2] and critical-path projection [12], [13] were implemented as baselines rather than cited.

Against commercial products, the position is complementary rather than competitive. Computer-vision monitoring systems address the sensing problem, generating milestone events automatically and with higher fidelity than manual recording [14]. They do not interpret those events [14], [15]. A system of the kind built here consumes exactly the stream such products emit, and the two would compose naturally: better sensing would improve the input to this system without altering its contribution. Products marketed directly to coordinators [15] remain, on the evidence of their published descriptions, oriented towards visibility and record-keeping rather than towards recommending action under a decision hierarchy.

5.4 Strengths and Limitations

The principal strength of the work is that its central claim is bounded. The signal–proof formulation predicts where an advantage should exist and where it should not, and the evaluation confirms both halves: the advantage is largest where a cause is observable long before its consequence, and is exactly zero for late inbound aircraft, where knowledge and proof coincide. A formulation yielding an advantage everywhere would indicate an artefact of construction rather than a property of the domain. A second strength is that safety behaviour is enforced structurally,

and the evidence that this matters is direct, since a language model given the hierarchy explicitly nonetheless emitted stand-down language on one occasion and was corrected by the layer [23], [25].

The limitations are substantial and are stated clearly. The evaluation is simulation-based, and although scenarios are sampled from real delay occurrences, no recommendation produced by this system has been acted upon in a live operation. The source data comprise 23 delay occurrences across 15 codes from a single operation, which is sufficient to ground a domain model but not to support statistical generalisation. Discovery points are assigned per delay code rather than per occurrence, which assumes a regularity that real operations will not exhibit. The recovery-rate parameter underlying the outcome result is an assumption tested by sensitivity analysis rather than a measured quantity. The system models a single stand in isolation, whereas a coordinator works several concurrently, and the sequencing problem that creates is not addressed. The expert appraisal, discussed further below, does not constitute independent validation. Finally, the reported-issue facility described in Section 3.9 was added after the evaluation was conducted and is therefore excluded from the reported results, a boundary maintained deliberately rather than blurred.

5.5 Future Work

Five directions follow directly from the limitations above. The first and most consequential is multi-turn sequencing. A coordinator holding several aircraft must decide which to attend to, and the decision is not simply a matter of taking the largest delay first: a recoverable ten-minute delay may warrant attention over an unrecoverable forty-minute one, precisely because the latter cannot be improved by presence. The signal–proof formulation extends naturally to this problem, since a cause that is knowable early is a cause that can be triaged early, and the resulting allocation problem is a natural application for an agentic planner [19], [20].

The second is integration with automated sensing. The system currently depends on manually recorded timestamps, whose granularity was sufficient to cause a spurious breach prediction until a materiality threshold was introduced. Computer-vision-derived events [14], [15] would remove that dependency and sharpen the signal times on which the advantage rests.

The third is prospective operational evaluation [38]. The strongest available evidence would be a shadow deployment in which the system's recommendations are recorded alongside the

coordinator's actual decisions, without being shown, allowing agreement and lead time to be measured against real operational outcomes rather than simulated ones.

The fourth is a properly constituted user study. The appraisal conducted here is formative, and a study recruiting coordinators with no involvement in the system's construction, separating assessment of recommendation quality from assessment of the interface, would establish what this project could only indicate. The fifth is learning the recovery model from the data [3], [7]. The recovery rate is presently an assumption; with sufficient records of interventions and their outcomes it could be estimated per delay code, which would replace the outcome chapter's most contestable parameter with a measured one.

5.6 Reflection

Three episodes during the project changed the work materially and are worth recording, since each was a failure before it was a finding.

The first concerned measurement. An early comparison between the rule and language backends produced agreement below chance, which was initially interpreted as evidence about model capability. It was not. The system prompt requested a judgement of recoverability without defining the term, and the two backends interpreted it differently, one against the clock and one against the cause, while the rule backend simultaneously derived its recoverability flag from a source that could contradict its own selected action. Two defects had combined to produce a result that appeared substantive. The lesson taken from these findings is that a surprising result should be treated first as a suspected fault in the instrument and that the interpretive burden of a term the system relies upon must be discharged explicitly rather than assumed shared.

The second concerned the boundary of the contribution. When the interface was extended to replay a turnaround minute by minute, the agent and the critical-path projector were found to identify problems at the same instant, eliminating the advantage the evaluation had measured. The cause was conceptual rather than technical: a timestamp constitutes proof, and proof becomes available to every method simultaneously. The advantage depends on receiving a cause before it materialises, which is routine in the operation but had not been represented as an input. Correcting the issue produced the clearest statement of what the project contributes, namely the capacity to ingest a class of information that a critical-path method has no vocabulary to express.

That the clarification arrived from an implementation difficulty rather than from analysis is instructive.

The third concerned the gap between building a system and its usability. The appraisal was conducted expecting to validate the recommendations and instead exposed three defects in them, including a delay code misclassified as unrecoverable because inability to repair a fault had been confused with inability to act upon it. A subsequent round with colleagues produced uniform agreement and no comments, which was initially disappointing and then more informative than agreement would have been: the respondents reported the wording as too complex to read at speed, and an instruction that must be decoded is one that will be skipped under load. The system had been written in the register of the person analysing the problem rather than in that of the person acting on it. All user-facing language was rewritten in consequence.

With hindsight, three decisions would be taken differently. Domain elicitation would be separated from implementation and completed first, since the corrections that arrived late in the project were all available at the outset had they been sought systematically rather than encountered incidentally. The user evaluation would be scheduled early enough to permit a second independent round after revision, rather than being attempted once the system was substantially fixed. And the evaluation harness would be treated as a deliverable subject to its own tests from the beginning, since the measurement defects described above were discovered by inspection rather than by any examination designed to catch them.

5.7 Conclusion

The research question asked what an agentic layer adds over classical baselines, and where it fails. It adds four minutes of mean detection lead over critical-path projection, cause attribution where the baselines offer none, and directed action on every breach including those from which no minutes can be recovered. It fails, predictably and by design, in situations where a cause becomes knowable only when it is provable, with late inbound aircraft serving as the clearest example. It cannot be relied upon to observe an operating hierarchy through instruction alone, which is why that hierarchy is enforced outside the model. And it has yet to be shown to help a coordinator in a live operation, which remains the evidence the work most needs and lacks.

References

- [1] ACI EUROPE, EUROCONTROL, IATA, and CANSO, "Airport collaborative decision making (A-CDM)," SKYbrary Aviation Safety. [Online]. Available: <https://skybrary.aero/articles/airport-collaborative-decision-making-cdm>. [Accessed: 01-Aug-2026].
- [2] EUROCONTROL, "EUROCONTROL specification for airport collaborative decision making (A-CDM)," Edition 1.0, Brussels, Belgium, 2025. [Online]. Available: <https://www.eurocontrol.int/publication/eurocontrol-specification-airport-collaborative-decision-making-cdm>. [Accessed: 01-Aug-2026].
- [3] X. Tang, J. Wu, C.-L. Wu, Y. Ding, and S. Zhang, "Dynamic prediction of aircraft turnaround milestone times using a cascaded gradient boosting model for improved airport collaborative decision-making," *Journal of Air Transport Management*, vol. 128, Art. no. 102842, 2025. doi: 10.1016/j.jairtraman.2025.102842.
- [4] M. Luo, M. Schultz, H. Fricke, and B. Desart, "Data-driven fusion of turnaround sub-processes to predict aircraft ground time," in *Proc. 25th Air Transport Research Society (ATRS) World Conf.*, Antwerp, Belgium, 24–27 Aug. 2022. [Online]. Available: https://www.unibw.de/lvk/publications/pdf/2022_atrs_turnaround.pdf. [Accessed: 01-Aug-2026].
- [5] D. Sahadevan, H. Al Ali, D. Notman, and Z. Mukandavire, "Optimising airport ground resource allocation for multiple aircraft using machine learning-based arrival time prediction," *Aerospace*, vol. 10, no. 6, Art. no. 509, 2023. doi: 10.3390/aerospace10060509.
- [6] A. Murad and M. Ruocco, "Pre-tactical flight-delay and turnaround forecasting with synthetic aviation data," *CEAS Aeronautical Journal*, 2026. doi: 10.1007/s13272-026-00941-7.
- [7] S. Alfarhood *et al.*, "Predicting flight delays with machine learning: A case study from Saudi Arabian Airlines," *International Journal of Aerospace Engineering*, 2024. doi: 10.1155/2024/3385463.
- [8] Q. Zhu, S. Chen, T. Guo, Y. Lv, and W. Du, "A spatio-temporal approach with self-corrective causal inference for flight delay prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 12, pp. 20820–20831, 2024. doi: 10.1109/TITS.2024.3443261.

- [9] J. Guo, J. Li, J. Yuan, Y. Yang, and Z. Ren, "A data-driven framework for flight delay propagation forecasting during extreme weather," *Mathematics*, vol. 13, no. 21, Art. no. 3551, 2025. doi: 10.3390/math13213551.
- [10] "Air travel delay prediction: Feature engineering and ML approaches," UC Berkeley School of Information, 2025. [Online]. Available: <https://www.ischool.berkeley.edu/projects/2025/air-travel-delay-prediction-feature-engineering-and-ml-approaches>. [Accessed: 01-Aug-2026].
- [11] J. Chen and M. Li, "Chained predictions of flight delay using machine learning," in *Proc. AIAA SciTech Forum*, 2019.
- [12] S. Birolini and A. Jacquillat, "Day-ahead aircraft routing with data-driven primary delay predictions," *European Journal of Operational Research*, 2023. doi: 10.1016/j.ejor.2023.02.035.
- [13] K. Kontodimou, D. Peteinatos, S. Bagioneta, and K. Kepaptsoglou, "Proactive aircraft turnaround buffer optimization integrating machine learning and scenario analysis," *Journal of the Air Transport Research Society*, 2026. doi: 10.1016/j.jatrs.2026.100151.
- [14] S. Yıldız, O. Aydemir, A. Memiş, and S. Varlı, "A turnaround control system to automatically detect and monitor the time stamps of ground service actions in airports: A deep learning and computer vision-based approach," *Engineering Applications of Artificial Intelligence*, vol. 113, Art. no. 105032, 2022. doi: 10.1016/j.engappai.2022.105032.
- [15] Assaia International AG, "Visibility to optimize the turnaround." [Online]. Available: <https://www.assaia.com/>. [Accessed: 01-Aug-2026].
- [16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [17] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [18] Y. Qin *et al.*, "Tool learning with foundation models," *ACM Computing Surveys*, vol. 57, no. 4, Art. no. 101, pp. 1–40, 2025.

- [19] K. Stein, D. Fišer, J. Hoffmann, and A. Koller, "Automating the generation of prompts for LLM-based action choice in PDDL planning," arXiv:2311.09830, 2023.
- [20] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of LLM agents: A survey," arXiv:2402.02716, 2024.
- [21] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. arXiv:2303.11366.
- [22] M. Renze and E. Guven, "Self-reflection in LLM agents: Effects on problem-solving performance," arXiv:2405.06682, 2024.
- [23] M. Cemri *et al.*, "Why do multi-agent LLM systems fail?" arXiv:2503.13657, 2025.
- [24] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, "Survey on evaluation of LLM-based agents," arXiv:2503.16416, 2025.
- [25] S. Agashe, Y. Fan, and X. E. Wang, "LLM-Coordination: Evaluating and analyzing multi-agent coordination abilities in large language models," arXiv:2310.03903, 2023.
- [26] S. Abdulhak, W. Hubbard, K. Gopalakrishnan, and M. Z. Li, "CHATATC: Large language model-driven conversational agents for supporting strategic air traffic flow management," in *Proc. 11th Int. Conf. on Research in Air Transportation (ICRAT)*, Singapore, Jul. 2024. arXiv:2402.14850.
- [27] L. Wang, J. Chou, A. Tien, X. Zhou, and D. Baumgartner, "AviationGPT: A large language model for the aviation domain," in *Proc. AIAA AVIATION Forum and ASCEND*, Las Vegas, NV, USA, 2024, Paper 4250.
- [28] S. R. Andrade and H. S. Walsh, "SafeAeroBERT: Towards a safety-informed aerospace-specific language model," in *Proc. AIAA AVIATION Forum*, San Diego, CA, USA, 2023, Paper AIAA 2023-3437. doi: 10.2514/6.2023-3437.
- [29] D. Nielsen, S. S. Clarke, and K. M. Kalyanam, "Towards an aviation large language model by fine-tuning and evaluating transformers," in *Proc. AIAA DATC/IEEE 43rd Digital Avionics Systems Conf. (DASC)*, 2024, pp. 1–5.

[30] "Aviation safety: Retrieval over ASRS incident reports," UC Berkeley School of Information, 2026. [Online]. Available: <https://www.ischool.berkeley.edu/projects/2026/aviation-safety>. [Accessed: 01-Aug-2026].

[31] A. Tikayat Ray, A. P. Bhat, R. T. White, V. M. Nguyen, O. J. Pinon Fischer, and D. N. Mavris, "Examining the potential of generative language models for aviation safety analysis: Case study and insights using the Aviation Safety Reporting System (ASRS)," *Aerospace*, vol. 10, no. 9, Art. no. 770, 2023. doi: 10.3390/aerospace10090770.

[32] "Large language models for aviation safety: Enhancing incident analysis through summarization of ASRS reports," University of Maryland. [Online]. Available: <https://www.cs.umd.edu/content/large-language-models-aviation-safety-enhancing-incident-analysis-through-summarization-asrs>. [Accessed: 01-Aug-2026].

[33] V. Siddeshwar, A. Azim, S. Alwidian, and M. Makrehchi, "Towards enhancing aviation safety through advanced incident analysis using large language models," in *Proc. 34th Int. Conf. on Collaborative Advances in Software and Computing (CASCON)*, 2024, pp. 1–7. doi: 10.1109/CASCON62161.2024.10837802.

[34] A. Iyengar, A. Tiselska, D. Samaraweera, and H. Liu, "Building trust in the skies: A knowledge-grounded LLM-based framework for aviation safety," in *Proc. IEEE SoutheastCon 2026*, 2026. doi: 10.1109/SoutheastCon63549.2026.11476130.

[35] F. M. Aguilera, "Multi-agent AI system for airline operation control with DeepSeek LLM," *Medium*, 2025. [Online]. Available: <https://medium.com/ai-simplified-in-plain-english/multi-agent-ai-system-for-airline-operation-control-with-deepseek-llm-a77a7a72c701>. [Accessed: 01-Aug-2026].

[36] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Human Factors*, vol. 39, no. 2, pp. 230–253, 1997.

[37] M. L. Cummings, "Automation bias in intelligent time critical decision support systems," in *Proc. AIAA Intelligent Systems Technical Conf.*, 2004. doi: 10.2514/6.2004-6313.

- [38] "Human-in-the-loop AI in automated airspace," Zenodo, 2026. doi: 10.5281/zenodo.19968129. [Online]. Available: https://indgr.org/uploads/1777700955_Human-in-the-Loop%20AI%20in%20Automated%20Airspace.pdf. [Accessed: 01-Aug-2026].
- [39] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [40] British Computer Society, "BCS Code of Conduct," BCS, The Chartered Institute for IT, Swindon, UK. [Online]. Available: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>. [Accessed: 01-Aug-2026].

Appendices

Appendix A: Project Proposal

The project proposal submitted at the outset defined the scope carried through to completion, with two substantive changes recorded in Chapter 5. The user evaluation was reduced from a multi-participant study to a single formative appraisal, and a reported-issue input channel was added to the interface after the evaluation had been conducted and is therefore excluded from the reported results.

The proposal set the aim as the design, implementation and evaluation of an agentic system for real-time turnaround delay attribution and recovery recommendation, against threshold-monitoring and critical-path baselines. That aim was met in full. The proposal is retained by the supervisory team and is available on request.

Appendix B: Project Management

The project was planned as eight work packages and tracked against a Gantt chart, revised twice as scope developed. The work packages covered domain elicitation, schedule modelling, agent implementation, baseline implementation, evaluation harness construction, expert appraisal, interface development and the written report.

Two deviations from the original plan are worth recording. Domain elicitation was interleaved with implementation rather than completed first, which is discussed in Section 5.6 as a decision that would be taken differently. Interface development was scheduled late, which compressed the time available for user evaluation and contributed to the limitation reported in Section 4.7.

Supervisory meetings were held throughout the project, with notes recorded and agreed actions carried forward. The implementation was version-controlled from the first working prototype, and the commit history in the repository cited in Appendix C serves as the detailed record of development, showing the sequence in which components were built, the points at which defects were identified, and the corrections made in response.

MSc AI Project — Gantt Chart

Agentic LLM Copilot for Aircraft Turnaround Monitoring & Delay Recovery
Period: 10 Jun – 1 Aug 2026 • ~12.5 h/week (part-time) • Deadline: 1 Aug 2026

#	Task	Type	Start	End	Wks	Est. hrs	9 Jun	16 Jun	23 Jun	30 Jun	7 Jul	14 Jul	21 Jul	28 Jul
1	Literature review & finalise scope	Core	10 Jun	20 Jun	1.4	18								
2	Build turnaround simulator (A-CDM milestones, dependency graph, disruption injection)	Core	16 Jun	29 Jun	1.9	23								
3	Implement baselines (rule-based + critical-path)	Core	23 Jun	4 Jul	1.6	20								
M	★ Simulator fidelity check (validate against domain knowledge)	Milestone	29 Jun	29 Jun	0	13								
4	Agent architecture: perceive–predict–plan–act loop + tool use	Core	30 Jun	13 Jul	1.9	23								
5a	Recovery recommendation logic	Core	7 Jul	16 Jul	1.3	16								
5b	Stakeholder comms drafting (extension)	Extension	16 Jul	22 Jul	0.9	13								
6	Evaluation experiments (N seeded disrupted turnarounds vs baselines)	Core	13 Jul	23 Jul	1.4	18								
7	Expert assessment of recommendations (extension)	Extension	21 Jul	27 Jul	0.9	13								
8	Analysis & dissertation write-up	Buffer	16 Jul	31 Jul	2.1	27								

Buffer week (28 Jul–1 Aug): proofread, final fixes, submission. Extensions (5b, 7) are cuttable if core slips.

Hours budget check

Total estimated hours (core + extensions):

184

Available hours (7.5 weeks × 12.5 h):

94

Legend

	Core (must finish)
	Cuttable extension
	Buffer / write-up
	Milestone

Figure 8 Project Gantt chart. Eight work packages tracked from domain elicitation through to the written report, revised twice as scope developed.

The mid-point review presentation is available on request.

Appendix C: Artefact/Dataset

The artefact comprises the agent implementation, two baseline implementations, the evaluation harness, the automated test suite and the demonstration interface, together with the de-identified dataset required to reproduce every reported result.

Repository (public, de-identified): <https://github.com/21seift/turnaround-copilot>

Repository (full, restricted access): [https://github.com/21seift/MSc Turnaround Project](https://github.com/21seift/MSc_Turnaround_Project)

The public repository contains the complete implementation, the automated test suite, the demonstration interface and the de-identified dataset sufficient to reproduce every result reported in Chapter 4. Access to the full repository is restricted, since the dispatch records from which the domain model was elicited are the commercial property of the operator concerned and are subject to contractual confidentiality, as set out in Section 1.4. The supervisory team may be added as collaborators immediately upon notification of a GitHub username or email address; the full repository differs from the public copy only by the inclusion of those confidential source records.

Structure. The package `turnaround_sim` contains the schedule model, rebasing arithmetic, milestone model, attribution layer, agent, reasoning backends, constraint layer, both baselines and the evaluation harnesses. Tests are in `tests`. Data is in `turnaround_sim/data`.

Running the system. The implementation requires Python 3.11, with SimPy as its runtime dependency and pytest for the test suite. The demonstration interface is served by the standard library and requires no framework:

```
pip install -r requirements.txt
```

```
python -m turnaround_sim.serve          # interface at localhost:8000
```

```
python -m turnaround_sim.demo          # single turn, agent working
```

```
python -m turnaround_sim.demo --safety  # hierarchy under a live hazard
```

```
python -m turnaround_sim.demo --compare # all three approaches, same turn
```

```
python -m turnaround_sim.evaluate_emerging # detection lead (Table 4.1, 4.2)
```

```
python -m turnaround_sim.evaluate_outcomes # recovered delay (Table 4.3)
```

```
python -m turnaround_sim.compare_reasoners # backend comparison, offline mock
```

```
pytest tests/ # full regression suite (13 tests)
```

Reproducibility. Scenario generation is seeded. Language-model responses are cached against a hash of model identifier, system prompt and request, so that evaluation runs are deterministic and incur no repeated inference cost. Confidential source data, credentials and cached responses are excluded from version control. The evaluation reads the de-identified derivative in preference to the confidential source and was verified to reproduce every figure reported in Chapter 4 with the confidential files absent.

Dataset. The de-identified derivative contains 23 delay occurrences, each recording a delay code, a duration, a discovery point and a short mechanism description, with all operator, airport, date and personnel references removed. It is included in the public repository. The confidential source records are not.

Appendix D: Screencast

A screencast demonstrating the system is available at the following link.

Link: [your URL]

The recording covers the demonstration interface accepting turnaround timestamps, the rebasing of the countdown from the in-block anchor, attribution of a developing delay to its cause, the projected door closure, and the resulting recommendation. It includes a comparison in which the agent identifies a refuelling delay while the critical-path baseline remains silent, and a safety case in which a hold is asserted with manage-around actions retained beneath it.

Appendix E: Appraisal Instrument

The instrument used for the appraisal reported in Section 4.6 presents all 27 evaluation cases, each showing the situation and the agent's recommendation, and asks the respondent whether the recommendation would be actioned as written, actioned with modification, or rejected, together with whether the recoverability classification is correct and what the respondent would do differently.

The instrument records no name, staff number or employer, transmits nothing automatically, and states the purpose of the study before the first case. Responses are exported by the respondent and returned manually.

Available at: <https://tranquil-cheesecake-d01e34.netlify.app/>

The hosted version is the revised instrument, incorporating the corrections described in Section 4.6 and the plain-language rewrite that followed the second appraisal round. It is generated from the evaluation case set by `turnaround_sim/export_appraisal.py` in the repository cited in Appendix C.